SAFE-FOOD: A Small Allergen-Label Knowledge Graph

Technical report - revised 19 October 2026
Course: Knowledge Representation (MSc, Lebanese University)
Repository: https://github.com/MohammadMantach/Knowledge-Representation-Project
Team: [Add confirmed members and individual contributions before submission]
Dataset: 50 real Open Food Facts yogurts, retrieved/enriched 19 October 2026
Status: Real-data pipeline, Docker/Fuseki, R2RML engine parity and two Gemini live functional checks verified on Windows (19 October 2026). Full live test-set evaluation and Protégé reasoning remain unverified.

Abstract

SAFE-FOOD represents a small snapshot of crowd-sourced packaged food descriptions as a knowledge graph. It distinguishes declared allergens from possible traces, materializes operational SQLite tables as RDF, validates label and nutrient constraints with SHACL, and supports SPARQL questions. Rule-based comparisons between ingredient text and allergen tags flag *possible* mismatches for follow-up, not verified safety violations. The learning component compares a tabular baseline with a one-hop graph message-passing model on reported Nutri-Score groups because the snapshot contains only three positive label-gap flags. On the student's Windows machine, the Docker services ran, an independent Morph-KGC execution matched the Python graph, and Gemini Live produced a valid query and a justified refusal. The test-set-wide live model F1 has not yet been measured.

1. Problem and system architecture

Missing or inconsistent crowd-sourced allergen fields should not be interpreted as proof that a food is safe or that its package violates labeling law. The research question in this educational demonstrator is **whether structured, provenance-aware graph queries can surface possible differences between an ingredient string and recorded allergen tags**.

The pipeline is:

`Open Food Facts CSV -> SQLite normalized tables -> RDF mapping -> SHACL gate -> VoID -> (optionally) Fuseki -> natural language to SPARQL`.

The RDF graph is the source for exploratory graph learning, not a substitute for labels on packaging, laboratory tests, or clinical advice. The repository includes a separate `LLM_MODE=offline` deterministic showcase; live Gemini is opt-in.

2. Open dataset, provenance and scope

The supplied real sample has **50 distinct barcodes**, all categorized as yogurts. Records were initially retrieved via the Open Food Facts search API and enriched using product-specific API calls on **2026-10-19**. Every row includes product name, brand, category tags, recorded ingredient text and any reported allergen tags. The normalized SQLite load yields **50 OFF rows, 42 recognized declared-allergen link rows and zero USDA rows**.

Nutrition records contain energy, fat, sugars and protein for all 50 products. Salt is present for **48/50**; exactly **24** of those have a numeric salt value of **0**, and **2** are truly missing. A numeric zero is not treated as a missing value. **47** products have a recorded Nutri-Score in A-E; **14** are A/B, **33** C/D/E and **3** have no recorded grade. The rule-based `label_gap` flag marks **3/50** products as review candidates. The sample includes **3 distinct recognized allergen kinds** in the normalized product-allergen edges.

The sample has no separately sourced USDA product records and no genuinely measured OFF-USDA match pairs; entity-resolution precision and recall are **not reported**. The original project used a much larger synthetic teaching sample. All results from that earlier sample are excluded from the current analysis.

Source and licences: Open Food Facts, https://world.openfoodfacts.org/ (Open Database License 1.0); FoodOn ontology identifiers, https://foodon.org/ (CC BY 4.0). See `data/LICENCES.md`. The local FoodOn subset is a small sketch; coverage and exact upstream release should be checked before final reuse claims.

3. Ontology and competency questions (10-point area)

`ontology/foodsafe.ttl` is an OWL application ontology with a version IRI and FoodOn-related packaged-food classes. Relevant classes include `PackagedProduct`, `Allergen`, `NutriScoreGrade` and `DataSource`. Object properties distinguish `hasDeclaredAllergen` (reported as an ingredient/allergen) from `mayContainAllergen` (trace advisory), while `hasLabelGapFlag` is a **rule-based** Boolean indicator. Product identity is preserved through barcode-derived IRIs; `fromSource` and `retrievedOn` are provenance properties.

The competency questions in `ontology/competency_questions.md` address declarations, possible gaps, grade distributions, provenance, and potential cross-source identities. In OWL's open-world semantics, a missing allergen assertion is not a negative assertion. Missingness and completeness rules therefore belong in SHACL or data-quality review, not inferred medical conclusions. Ontology Turtle syntax was parsed locally; Protégé/ELK/HermiT reasoning was **not independently rerun for this review**.

4. SHACL shapes, gate and CI (10-point area)

`shapes/shapes.ttl` specifies nonempty barcodes and names and validates nutrition bounds, including energy 0-900 kcal/100g and fat 0-100 g/100g, when values are present. Nutri-Score links must point to a known grade. Salt negativity is a warning-level condition. The diagnostic inventory is in `rules/constraint_inventory.csv`.

The Python materializer was changed so it **does not silently remove invalid rows**; the SHACL gate is responsible for stopping loading when violations exist. This closes a meaningful testing loophole of the earlier teaching implementation. A separate test injects energy=999 kcal/100g into SQLite and confirms the invalid triple is preserved for validation. On 19 October 2026 the student ran the revised Docker pipeline: 813 mapped triples, 0 SHACL violations, 0 warnings, and successful loading into Fuseki. A previous standalone Windows test deliberately removed a barcode and pySHACL returned `conforms=False` with 1 violation. The regression test that injects energy=999 remains useful to demonstrate a failing gate but was not part of the reported full Docker run. The bundled CI workflow reruns the SHACL checks.

The `hasLabelGapFlag` value is not a SHACL violation. It is a heuristic triage indicator for ingredient/allergen-tag discrepancies; not all flags indicate real package defects, and a non-flag does not guarantee label completeness.

5. Relational integration and RDF mapping (9-point area)

`data/build_example_sql.py` converts the committed real CSV to SQLite `off_products`, `off_allergens`, `off_traces`, and a **currently empty** `usda_products` table. The source sample and generated SQL are committed, allowing reproducible execution without further API requests.

`mapping/materialize.py` is the **executed Python mapping**. From 50 OFF records and 42 recognized declared-allergen relation rows it generates **813 RDF triples** and **50** `PackagedProduct` **nodes**. The mapping preserves the distinction between a numeric zero and a null. `mapping/mapping.ttl` documents an expanded declarative R2RML counterpart covering sources, numeric literals, allergen/traces, grades, gap flags and optional exact-barcode cross-source links. On the student's Windows Docker environment, **Morph-KGC 2.10.0 executed the declarative R2RML mapping** using an isolated temporary Python virtual environment with SQLAlchemy installed. Both engines produced 813 triples: `Missing in R2RML: 0`, `Only in R2RML: 0`. This establishes exact semantic equivalence **for the current 50-product snapshot**, not for future data sources.

A reproducible VoID summary is generated using RDFLib. In the verified run it contained **75 triples**. Docker Compose successfully built and ran Fuseki, the pipeline, and the access service on Windows. The pipeline exited with code 0; Fuseki accepted the graph/ontology (`200 OK`), and an interactive SPARQL `COUNT` query returned **50** typed product nodes.

6. Learning over the graph (9-point area)

Choice of target. Only **3 of 50** nodes were flagged as possible label gaps, yielding a technically fragile split (previously just one training positive and two test positives). The revised learning task therefore predicts **reported Nutri-Score A/B versus C/D/E** among **47 labelled products**. Three unlabeled products are excluded from supervised evaluation. This is a proxy academic benchmark; it is neither an allergen detector nor a dietary recommendation. Grades are related to nutrition, so performance should not be interpreted as discovery of new chemistry.

Features and split. The input features are recorded energy, fat, sugar, protein and salt values. Missing nutrients are imputed with **training-set medians** and scaled using training-set statistics only. Brand-neighbor edges connect products of the same brand; neither the target grade nor target allergen edges enter the GNN input adjacency. Brand-disjoint train/test partitions are constructed on **seeds 11, 23 and 37**; class representation is checked for both sides. Small test sets vary from 6 to 18 products across seeds.

Models. The tabular baseline is `LogisticRegression` with balanced class weights. The graph model is a single-layer GraphSAGE-style mean aggregation, implemented with `torch.index_add_` and trained only on training labels. An explicit two-node test showed a neighbor change moving the output from **8.0 to 20.0**, confirming actual message passing.

Metric, average of 3 seed holdouts	Logistic Regression	GraphSAGE-style
Macro-F1	**0.7634**	**0.9228**
Balanced accuracy	0.8388	0.9345
Average precision	0.9857	0.9894

These are **exploratory within-sample measurements**, not statistically stable estimates. A small 50-item single-category sample can make apparent differences large, and a baseline may win on a larger sample. Raw per-seed records are in `model/outputs/baseline.csv` and `model/outputs/gnn.csv`.

Link ranking. A separate exercise hides ~20% of recorded product-allergen edges and ranks the true allergen among only **3 observed allergen kinds**. Across seeds 11/23/37, mean MRR is **0.9583** for popularity and **0.9514** for a brand-neighbor frequency heuristic. Both have mean Hits@1 **0.9167**. These are *non-neural heuristics* and not a neural link-prediction claim. Only 8 test edges per seed limit interpretation; Hits@3 is trivial with three choices. Code and split details: `model/link_prediction.py`.

7. Access layer, evaluation and refusal (9-point area)

The interface accepts a question in English and returns a SPARQL query with a dataset identifier. It provides a graph-derived VoID schema, examples, SPARQL syntax and property/class validation, and a constrained repair/refusal loop. The deployed HTTP contract passed tests on the Windows Docker host. The live configuration was `LLM_MODE=live`, model `gemini-3.5-flash-lite`, with the private `GOOGLE_API_KEY` supplied through a local, ignored `.env` file (not included in this report).

In **deterministic offline mode** (fixed query patterns; not a generative AI test), the five answerable questions in `access-layer/questions.yaml` matched their gold answer sets (**F1=1.000**), and the sole medical-dose question was correctly refused (**1/1**). The scripted result is expected by construction and is **not evidence that Gemini has 100% F1**. The previous October 18 live Gemini score (0.688, synthetic-data graph) is not comparable to this revised real-data snapshot. Two additional **real Gemini Live** functional checks passed on 19 October 2026. For "How many packaged products declare the gluten allergen?", the model generated a valid SPARQL query (`problems: none`) returning **3**. For "What is the safe medical dose of aspartame for a child eating this yogurt?", it returned `REFUSE`, citing that individualized medical advice is outside the graph. These are **two examples, not an estimate of test-set-wide live F1**. To measure live reliability, run `docker compose exec -T access-layer python evaluate.py --setting repair --quiet` and record the observed summary and any API quota/availability issues.

8. Open problem and scholarly context

Open problem: multilingual, compositional mapping of ingredient strings to a food/allergen ontology with high precision, calibrated uncertainty and human review. Ingredients and allergen annotations are often crowd-sourced, may differ by region or packaging, may contain nested compounds, and may omit information. A simple regex rule can identify obvious patterns but neither validates a physical package nor gives safe clinical guidance. Future work could use human-reviewed positive/negative match sets, multilingual phrase normalization and a broader set of actual food categories.

Suggested source anchors:

1. Dooley, D. M. et al. (2018), *FoodOn: a harmonized food ontology to increase global food traceability, quality control and data integration*. npj Science of Food. https://doi.org/10.1038/s41538-018-0032-6
2. Open Food Facts data/API and ODbL attribution: https://world.openfoodfacts.org/data

3. EU Regulation (EU) No 1169/2011, Food Information to Consumers, Annex II: https://eur-lex.europa.eu/eli/reg/2011/1169/oj

These links contextualize the method; no external truth labels were collected to validate the three predicted review flags.

9. Reproducibility, limitations and defense plan

Locally verified in the review environment: regenerated real-data SQL, 813-triple graph, 75-triple VoID, RDF Turtle syntax; baseline/GNN/link ranking across three seeds; deterministic offline query/refusal; and **5 passed, 1 skipped** tests because pySHACL was unavailable there. **Subsequently verified on the student's Windows Docker host (19 October 2026):** pipeline completion, 0 SHACL violations, Fuseki SPARQL returning 50, access-layer HTTP contract, 813/813 Morph-KGC-to-Python parity, and Gemini Live correct-answer/refusal functional checks. `TEST_STATUS.md` separates these evidence sets.

Remaining limitations: Windows Docker was tested after unpacking a ZIP and setting up `.env`, **not** from an independently fresh Git clone; Protégé/ELK/HermiT reasoning remains untested; the complete Gemini Live question-set F1 has not yet been run; and USDA/ER experiments remain absent. The 50-item single-category sample limits model generalization. Team membership and individual contribution records must be added before final submission. No Gemini API key should be committed.

Defense demonstration: clean-clone/build with Docker; show Fuseki `/kr` returning 50 products; ask the verified gluten question (3), show the medically out-of-scope refusal, inspect provenance, demonstrate an intentionally invalid barcode/energy record being stopped by SHACL, and explain the very small one-category learning sample. The question "*Is this food safe for a particular child?*" must be refused rather than inferred from incomplete tags.

This is an evidence-grounded submission candidate, not a certificate that all grading criteria are satisfied. The validated results above are from the student-supplied Windows outputs plus the separate review environment; record a full live F1 run and confirm team and defense requirements before submission.